

M1.1: K8s 容器 OOMKilled 根因与定位

Theory: Linux 内核 OOM Killer 机制通过评估 `/proc/[pid]/oom_score` 选定内存占用最恶劣的容器进程予以强杀, K8s 监测到 Exit Code 137 后立即标记 OOMKilled 并重建 Pod。

Details: 查看宿主机 `/var/log/messages` 确认 'Out of memory: Kill process' 记录; 使用 `'kubectl describe pod'` 在 Container Status 的 Last State 检索 'OOMKilled' 与 'Exit Code: 137'。调大 `containerspec` 下的 `'resources.limits.memory'`。

Trade-off: 盲目调大 limit 会降低单节点容器承载密度并引发内存超卖, 应采用 VPA (Vertical Pod Autoscaler) 进行合理基准推荐。

M1.2: K8s Pod 启动 CrashLoopBackOff 排错流

Theory: 容器内主进程 (PID 1) 退出码非 0 或健康检查持续失败, 触发 K8s 控制面以指数级退避延迟 (10s 至 5m) 不断尝试拉起容器。

Details: 使用 `'kubectl logs <pod-name> --previous'` 抓取前一次死掉的控制台 stdout 日志; 排查 'ENTRYPOINT' 执行权限及配置错误; 校验退出码 (如 Exit Code 1 代表应用运行时逻辑报错崩溃, 127 代表找不到对应物理命令)。

Trade-off: 发生退避时 Pod 短暂离线, 需配置合理的就绪探针 (Readiness Probe) 使网关自动摘除流量, 以防客户端请求打到黑洞。

M1.3: Pod Pending 调度受阻根因排查

Theory: K8s Scheduler 经过 Predicates (预选) 阶段评估, 在当前集群内找不到任何一个能完美满足 Pod 申请资源或亲和性约束的空闲节点。

Details: 运行 `'kubectl get events --sort-by=.metadata.creationTimestamp'` 排查 'FailedScheduling' 记录; 重点确认是 'Insufficient cpu/memory' 资源不足, 还是 Node 污点 (Taints) 与 Pod 容忍度 (Tolerations) 冲突。

Trade-off: 可以启用 cluster-autoscaler 触发底层云服务器弹性自动扩容节点, 但这会带来不可预测的费用抖动。

M1.4: K8s 运行期 ImagePullBackOff 故障排错

Theory: 容器运行时 (CRI) 在拉取镜像时遭遇证书错误、网络超时或公网镜像源 (如 Docker Hub) 匿名拉取频次限流, 导致拉取阻断。

Details: 执行 `'kubectl describe pod'` 检索 'ErrImagePull' 报错; 为私有库配置专有的 'imagePullSecrets'; 在企业内网中搭建专有的镜像缓存代理 (Harbor/Registry)。

Trade-off: 生产环境严禁直接依赖公共 Hub 镜像源, 核心镜像必须全部归档于内网可控镜像存储库中以防单点故障。

M1.5: 临时存储超限导致的容器被驱逐

Theory: 容器向本地挂载的 emptyDir 卷或根目录 '/tmp' 疯狂写入临时数据, 超出 'ephemeral-storage' 的 limit 限额, 被 Kubelet 强行杀死驱逐。

Details: 在 containerspec 下限制 'resources.limits.ephemeral-storage'; 排查未做切分的应用程序日志写盘逃逸; 配置 logrotate 将本地日志重定向, 或者挂载专有的物理 PV/PVC。

Trade-off: 限制临时存储可强迫研发优化 IO 动作, 但会对跑大文件缓存的旧系统造成崩溃隐患, 需结合内存型 tmpfs 卷缓冲折中。

M1.6: Pod 内存/磁盘压力驱逐 (Evicted) 自愈

Theory: 当节点物理内存可用率低于 100Mi 或文件系统剩余空间低于 10% 时, Kubelet 会为了维持宿主机 OS 稳定而主动杀掉低优先级 Pod 释放资源。

Details: 检测节点状态: `'kubectl get nodes'` 显示 'MemoryPressure' 或 'DiskPressure' 为 True; Kubelet 自动清除无用镜像; 根据 Pod 的 QoS 级别 (Guaranteed > Burstable > BestEffort) 确立首要驱逐目标。

Trade-off: 将所有 Pod 设为 Guaranteed 能获得最强生存力, 但这会使 Scheduler 锁定大量闲置资源额度, 极易引发资源铺张浪费。

M2.1: CoreDNS 解析延迟与 ndots search 冗余

Theory: K8s 容器默认采用 'ndots:5', 查找任意外部域名时都会按 'search' 路径逐级拼上 'svc.cluster.local' 发送多轮无效 DNS 查询, 拖垮 CoreDNS 性能。

Details: 容器内抓包: `'tcpdump -i any port 53 -nn'` 查看大量后缀冗余 of A 记录查询; 优化 'dnsConfig' 将 ndots 改为 1, 或者在外部域名后强行拼上绝对点后缀 (例如 'google.com.') 避开 search 路径查找。

Trade-off: 压低 ndots 后会导致同集群内跨 namespace 服务的短域名直接寻址失效, 必须在代码中规范使用全限定域名 (FQDN)。

M2.2: TCP SYN Flood 攻击与 Backlog 队列打满

Theory: 恶意连接只发送 SYN 包而不响应第三次 ACK 握手, 使操作系统的半连接队列 (SYN Queue) 或全连接队列 (Accept Queue) 迅速被无效套接字塞满, 抛弃正常流量。

Details: 检查丢包统计: `'netstat -s | grep "LISTEN"':` 修改内核参数: 启用 `'net.ipv4.tcp_syncookies = 1'`, 调大 `'net.ipv4.tcp_max_syn_backlog'` 与 `'net.core.somaxconn'`。

Trade-off: 调大 backlog 与启用 syncookies 能有效应对中轻度流量冲击, 但在高压 DDoS 下仍需使用云端 DDoS 高防服务进行边缘网络清洗。

M2.3: Connection Reset by Peer (RST) 根因排查

Theory: TCP 接收端收到与当前握手状态不匹配的报文, 或云端安全网关/物理防火墙因连接长时间空闲超时而主动向两端透传 RST 拆除连接。

Details: 捕获 RST 报文: `'tcpdump -i any "tcp[tcpflags] & tcp-rst != 0"':` 检查 `'/proc/sys/net/netfilter/nf_conntrack_max'` 确认 conntrack 表是否打满溢出; 检查应用端是否在未读完 socket 缓冲区时强行 close 产生 RST。

Trade-off: conntrack 大小受限于系统内存, 需要在客户端建立合理的连接保活 (TCP Keep-Alive) 与退避重试 (Backoff Retry) 以规避物理 RST 导致的雪崩。

M2.4: Nginx 502 Bad Gateway 经典故障排错

Theory: Nginx 网关作为代理端, 在连接上游应用 (PHP-FPM/Java/Go) 物理套接字时被拒绝, 或应用进程挂起死锁导致未完成 TCP 三次握手。

Details: 查阅 Nginx 错误日志 'nginx/error.log' 中的 'connect() failed (111: Connection refused)'; 检查上游服务端口存活状态及进程数上限 (如 PHP-FPM 的 max_children); 核查 Unix Socket 的读写权限。

Trade-off: 502 代表上游应用不可用, 网关应配置 'error_page 502' 快速短路返回静态缓存页以降低用户焦虑。

M2.5: Nginx 504 Gateway Timeout 超时故障排错	M3.2: PV/PVC Mount 卡死故障定位	M3.5: Vault 动态密钥引擎租约失效灾难修复	M4.2: ELK/Fluentd 收集端背压卡死与队列饱和
Theory: Nginx 成功与上游应用建立了 TCP 连接, 但上游由于慢 SQL、长循环或死锁卡死, 在规定的超时期限内未向网关写回任何字节。 Details: 日志显示 ‘upstream timed out (110: Connection timed out)’; 修改 Nginx 网关配置延长 ‘proxy_read_timeout’ 与 ‘fastcgi_read_timeout’; 在后端定位并消灭慢 SQL。 Trade-off: 盲目拉长 read timeout 会使 Nginx 积压大量挂起连接, 耗尽可用工作线程, 建议在后端推行大计算接口的异步化解耦设计。	Theory: 容器重建重新调度时, 网络共享存储卷 (如 AWS EBS 或 Ceph) 未能在原 Node 彻底 Detach 卸载, 导致新 Node 对卷发起 Mount 挂载被独占锁驳回, 引起 Multi-Attach 卡死。 Details: 查看 Events 报错 ‘Multi-Attach error for volume’; 登录 CSI 控制节点分析挂载状态锁; 检查存储后端设备是否写满导致 CSI 驱动自愈挂起。 Trade-off: 可通过管理员接口强行 Detach 旧节点上的物理卷来解脱卡死, 但有损坏未刷脏数据的物理风险。	Theory: Vault 为业务生成的数据库临时密码绑定了 Lease TTL 租约。若客户端由于 GC 停顿或网络抖动未能在租约到期前发起 Renew 续期, Vault 会从底层数据库中强制删掉该密码。 Details: 检查 Vault 审计日志确认密码撤销 (revocation) 记录; 检查应用端 SDK 内置续约轮询协程; 准备降备急救包; 管理员通过特权生成硬编码只读账号紧急装载。 Trade-off: 动态密码极大地规避了泄密风险, 但由于将 Vault 视为数据库连的关键依赖路径, 必须确保 Vault Cluster 的高可用。	Theory: 当 Elasticsearch 后台发生 GC 延迟或物理 IO 瓶颈导致写入性能突降时, 收集端 (Logstash/Fluentd) 的内存缓冲队列瞬间饱和, 反向触发背压, 使上游应用调用日志打印时发生阻塞卡顿。 Details: 配置收集器参数 ‘queue.type: persisted’ 将内存队列切换为磁盘持久化队列以承接流量洪峰; 检查 ES 写入端丢出的 ‘429 Too Many Requests’; 调大 ES 的 bulk 写入大小和索引线程池。 Trade-off: 磁盘队列能延缓灾难, 但若 ES 长时间未恢复, 依然会导致本地日志卷被完全写爆引发级联宕机。

M2.6: Cilium eBPF 容器网络重定向失败排查	M3.3: 云端元数据服务 (IMDSv2) 访问限制	M3.6: 云存储 Bucket 越权公开事件防御
Theory: Cilium 利用 SOCKMAP 机制重定向 TCP 套接字, 当 NetworkPolicy 网络策略配置错误或 Linux 内核 SOCKMAP 功能版本不匹配时, 会导致数据包在 BPF map 寻址失败丢弃。 Details: 运行 ‘cilium monitor -type drop’ 直观排查内核级丢包位置; 检查 Cilium Agent 与主机内核通信; 检查 ‘bpftool map dump’ 确认 SOCKMAP 条目是否缺失。 Trade-off: 故障时可降级至传统的 veth-pair 并发路由, 但网关网络层吞吐将显著降低。	Theory: 容器访问云厂商 169.254.169.254 获取 IAM 授权。IMDSv1 易遭受 SSRF 攻击泄露凭证, 新版 IMDSv2 强制使用 Token 认证, 并限制网络 Hop Limit 跳数 (防止穿透容器网卡)。 Details: 发起元数据请求时如果抛出 401, 需改用 HTTP PUT 握手换取 ‘X-aws-ec2-metadata-token’; 当容器在 Bridge 网络模式下运行, 需要把元数据网卡跳转步数调大于 1 (‘http-put-response-hop-limit=2’)。 Trade-off: 强制启用 IMDSv2 会导致未升级 SDK 的遗留容器无法获取云凭证, 应在迁移前对容器网桥逐一审计。	Theory: 运维人员配置 Bucket Policy 或 ACL 时, 错误地将 ‘Principal’ 设为 ‘*’ 通配符且 ‘Action’ 包含 ‘GetObject’, 使私有机密文件完全暴露于公网。 Details: 使用云原生安全配置审计系统 (如 CloudTrail、AWS Config) 检索包含越权公开的策略文件; 在账户级别一键开启 Block Public Access 物理防火墙; 使用私有 Bucket 配合 App 签名服务生成短效预签名 URL (Pre-signed URL)。 Trade-off: Bucket 严格私有会拉长前端访问静态静态图片和资源的延迟, 需要使用 CDN 配备专有签名缓存密钥隔离。

M3.1: Terraform 状态冲突锁死 (State Lock)	M3.4: VPC Peering 路由冲突与不对称路由	M4.1: Prometheus TSDB 写入路径高基数瓶颈
Theory: Terraform 执行部署计划时, 为防止多人并发篡改资源引起竞态, 会在后端 (如 S3/Consul) 加锁, 若部署进程被中途强杀, 锁将无法释放, 锁死后续 CI/CD。 Details: 报错提示 ‘Error acquiring the state lock’; 确认后使用 ‘terraform force-unlock <Lock-ID>’ 强行卸载旧锁; 执行 ‘terraform plan -detailed-exitcode’ 对比物理云资源与本状态差异。 Trade-off: 强行解锁存在脏写风险, 必须配合部署流水线的排他锁硬限制防止并发解锁。	Theory: 跨账号建立 VPC 对等连接时, 若两端 VPC 的 CIDR 网段重叠, 会导致本地路由优先级冲突; 或者路由表单向配置遗漏, 包发得出却回不来, 形成不对称路由。 Details: 检查 VPC 路由表的 CIDR 范围, 确保无子网覆盖; 使用 AWS Reachability Analyzer 生成端到端网络路径校验图; 使用 ‘mtr’ 跟踪 ICMP 在对等连接网关处的路由节点。 Trade-off: 在网络规划初期必须通过 IPAM (IP 地址管理) 进行网段全局排他分配, 严防网段重叠, 重叠网段只能采用复杂的 NAT 方案。	Theory: 在指标中错误引入了高 Cardinality 的动态 Label (例如用户 ID、物理端口或时间戳), 导致 Prometheus 需要为每个变化的 Label 实例化一条时间序列, 吃光物理内存并卡死 TSDB。 Details: 通过 ‘/api/v1/status/tsdb’ 抓取基数占比最高的热点 Label 与 Metric; 监控 ‘prometheus_tsdb_head_series’ 指标; 在 Prometheus 采集配置中通过 ‘metric_relabel_configs’ 使用 ‘keep’ 或 ‘drop’ 丢弃高基数 Label。 Trade-off: 丢弃 Label 会使应用层无法进行细粒度的聚合查询, 因此对大流量动态追踪场景应选用基于 Jaeger/Elasticsearch 的链路追踪, 而非 Prometheus 监控。

M4.3: 分布式链路上下文传递丢失与 Traceparent 剥离

Theory: 跨服务 RPC 或异步队列消费时，由于网关或中间件剥离了 HTTP Headers 中的 W3C ‘traceparent’ 头，或者异步多线程下未能传递 ThreadLocal 上下文，导致 Trace ID 断链。

Details: 验证反向代理或 Ingress 配置，确保未剥离 ‘traceparent’；使用 ‘OpenTelemetry’ 客户端 SDK，在异步线程池分发时使用 ‘Context.current().wrap(runnable)’ 包装线程执行实体。

Trade-off: 全量追踪上下文会带来内存与序列化性能损耗，在超大规模 QPS 系统下建议配置 0.1% 至 1% 的采样率限流。

M4.4: Grafana 大时间范围慢查询与 TSDB 扫描优化

Theory: 用户在 Grafana 面板检索数月大跨度指标时，由于未配置 step 步长，迫使 Prometheus 扫描数十亿样本点，瞬时吃光系统可用内存引发 OOM。

Details: 在 Grafana 慢查询中发现未配置动态步长；在 Query 中强行配置 ‘step = \$__interval’ 促使看板随范围自动粗化数据；在 Prometheus 侧为高频大查询定制录制规则（Recording Rules）预聚合数据。

Trade-off: 录制规则会牺牲历史指标的微观细节，但能使几十天的长趋势看板加载速度提升几十倍。

M4.5: Alertmanager 告警抖动、合并与去重调优

Theory: 告警风暴会使运维人员疲劳进而忽略致命警报，且基础设施短时物理中断会导致告警反复恢复/触发 (Flapping)。

Details: 配置 ‘alertmanager.yml’: ‘group_by: [‘alertname’, ‘cluster’, ‘service’]’ 进行同类告警合并；配置 ‘group_wait’(等待 30s 以便聚合后发)；调大 ‘group_interval’ 与 ‘repeat_interval’(如 12h) 防止重复轰炸。

Trade-off: ‘group_wait’ 设得太大延迟核心运维对致命故障的感知时机，对 P0 级别致命告警应绕过合并策略直接派发。

M5.1: SRE 错误预算烧钱率告警设计

Theory: 绝对值告警易引发误报，SRE 烧钱率 (Burn Rate) 能够监控错误预算消耗的加速度，即根据 SLO 预算的消耗速度动态预警，极大压缩响应时延。

Details: 编写 Prometheus 复合规则，使用 1 小时与 6 小时双时间窗口计算 Burn Rate；当 1 小时 Burn Rate > 14.4 (意味着在 1 小时内消耗了 30 天预算的 2%) 时，触发 P0 级即时电话告警。

Trade-off: 烧钱率计算依赖历史稳定指标基准，对于流量呈现极强潮汐性、波谷波峰悬殊的应用，需要对计算分母进行动态归一化。

M5.2: 混沌工程注入网络丢包与 CPU 限制

Theory: 通过在测试或预发环境主动注入网卡丢包延迟或打满 CPU 核心等故障，强力验证系统容灾冗余与 HPA 自动扩容策略是否真正能在实战中生效。

Details: 部署 Chaos Mesh 实例；下发 Chaos 实验：利用 Linux TC (‘traffic control’) 注入容器网卡 15% 丢包延迟；使用 ‘stress-ng’ 打满 CPU 物理核心，监控 HPA 在 3 分钟内拉起新 Pod。

Trade-off: 演练存在引发级联崩溃风险，演练计划必须配置 Blast Radius Control (爆炸半径红线) 并提供一键中止恢复 (Emergency Stop)。

M5.3: 服务分级降级与特性开关 (Feature Flag) 兜底

Theory: 极端流量洪峰（如大促）下，应用因下游慢服务导致工作线程池打满，必须通过特性开关一键关闭非核心依赖（如个性化推荐、足迹），短路返回静态数据。

Details: 接入 Feature Flag 引擎；配置基于 Spring Cloud Circuit Breaker 或 Go resilience 的降级 Fallback；将非核心 RPC 接口包装为软失败 (Soft Fail)，确保主交易环路通畅。

Trade-off: 降级会损害用户的使用体验，需要在系统设计初期明确梳理出业务分级拓扑，严防降级逻辑写死导致的核心逻辑损毁。

M5.4: 数据库秒级 PITR 崩溃恢复校验

Theory: 当面临敲诈勒索删库或人为重大误删时，通过全量物理备份配合增量 Write-Ahead Log (WAL) 归档，可以无缝回放并重建数据库到灾难发生前的任意秒级时间点。

Details: 验证 Postgres ‘pg_backrest’ 全量备份链；编写 ‘restore_command’；指定 ‘recovery_target_time = ‘2026-06-18 20:00:00+08’’，挂载 WAL 文件开始日志回放，在新实例上完成数据一致性审计。

Trade-off: 大体积数据库的 WAL 历史日志回放需要消耗海量 IO，回放过程可能持续数小时，必须建立日常的自动化恢复校验演练。

M5.5: SRE 无指责复盘 (Blameless Post-Mortem) 规范

Theory: 分布式系统故障是复杂态下的必然，惩罚个人非但不能防范故障，反而会导致员工隐瞒真实缺陷，只有坦诚剖析系统协作及架构漏洞，才能使系统实现长期演进。

Details: 编写事故 Timeline 挂载复盘大图，客观梳理：‘异常触发点’ ‘告警触发’ ‘SRE 介入’ ‘熔断降级止血’ ‘恢复服务’；运用 5 Whys (五个为什么) 多维下钻系统根因；制定具体的 Action Items，责任到人并规定完成期限。

Trade-off: 复盘能有效防范系统故障重演，但需要公司治理层具备极强的信任文化，谨防复盘沦为走过场的政治大会。

Zone T1: SRE Firefighting CLI & Confgs

kubectl logs, CoreDNS resolv.conf, somaxconn backlog, CSI Attach/Detach, IMDSv2 Hop Limit, PromQL Burn Rate, Chaos Mesh

Zone T2: System Faults & Diagnostics

OOMKilled_137_memory_saturation, CrashLoopBackOff_entrypoint_permission, CoreDNS_ndots_redundant_query, Nginx_502_504_gateway_timeout, CSI_volume_multi_attach_lock, Vault_lease_revocation_token, Prometheus_TSDB_series_cardinality, SRE_error_budget_blameless_post_mortem